

Absolutely 👍 Let's pick up **exactly** from Lesson 3.3.

## Lesson 3.3 — `<Link>` & Client-Side Navigation Under the Hood 🚀

The key question is:

**When I click a Next.js `<Link>`, what actually happens compared with clicking a normal `<a>` ?**

And importantly, we'll connect this to your earlier confusion about **React + Parcel vs Next.js**.

---

### 1. First: the normal `<a>` case

Imagine you're on:

/analyze/example.com



and you have:

```
<a href="/analyze/google.com">
  Google
</a>
```

You click it.

The browser sees:

```
<a href="/analyze/google.com">
```

and says:

"This is navigation to another document."

So roughly:

```
Click <a>  
  ↓  
Browser navigation  
  ↓  
GET /analyze/google.com  
  ↓  
Server sends a new HTML document  
  ↓  
Browser parses HTML  
  ↓  
Downloads required JS/CSS  
  ↓  
JS executes  
  ↓  
React starts/hydrates  
  ↓  
New page is displayed
```



## The important part

The **browser** is performing a document navigation.

The current document is replaced by another document.

---

## 2. What about React + Parcel?

This is where things get interesting.

In a traditional React + Parcel SPA, you might have:

```
<a href="/analyze/google.com">
```

If you use a normal `<a>` , the browser still does the normal navigation above.

React doesn't magically change what `<a>` means.

That's why React applications commonly use a routing library such as React Router:

```
<Link to="/analyze/google.com">  
  Google  
</Link>
```

React Router intercepts the click.

So instead of:

```
Browser  
↓  
new document request
```



you get something more like:

```
Click  
↓  
React Router intercepts  
↓  
history.pushState()  
↓  
React changes which component is rendered  
↓  
Browser does NOT reload the document
```



So React Router gives your SPA **client-side navigation**.

---

### 3. Now Next.js `<Link>`

Next.js has its own router.

So you write:

```
import Link from "next/link";  
  
<Link href="/analyze/google.com">
```

```
Google  
</Link>
```

When you click it, Next.js can intercept the navigation.

Conceptually:

```
Click <Link>  
  ↓  
Next.js Router  
  ↓  
Doesn't perform normal full document navigation  
  ↓  
Gets the necessary route information  
  ↓  
Updates the URL  
  ↓  
Updates the React UI
```

And this is where Next.js becomes more sophisticated than simply saying:

"We'll just render another React component."

## 4. Let's use YOUR analyzer example

Suppose your application has:

```
/analyze/example.com
```

and:

```
/analyze/google.com
```

Your UI might look like:

|                      |          |
|----------------------|----------|
| Performance Analyzer | ← Layout |
| example.com          | ← Page   |
| Performance: 82      |          |

Then you click:

Analyze Google



which navigates to:

/analyze/google.com



## 5. Does Next.js throw away the entire UI?

No.

This is one of the really important ideas in the App Router.

Suppose your structure is:

```
app/  
├─ layout.tsx  
└─ analyze/  
    ├─ layout.tsx  
    └─ [domain]/  
        └─ page.tsx
```



For:

/analyze/example.com



Next.js has something conceptually like:

```
Root Layout  
└─ Analyze Layout  
    └─ example.com Page
```



When you navigate to:

/analyze/google.com



the tree becomes:

```
Root Layout  
└─ Analyze Layout
```



└─ google.com Page

Notice something?

|                |         |
|----------------|---------|
| Root Layout    | SAME    |
| Analyze Layout | SAME    |
| Page           | CHANGED |



That's the magic.

## 6. So what does "preserve layouts" actually mean?

Imagine your layout contains:

```
export default function AnalyzeLayout({ children }) {  
  return (  
    <>  
      <Navbar />  
      <Sidebar />  
  
      {children}  
    </>  
  );  
}
```

And your page is:

```
export default function AnalyzePage() {  
  return <PerformanceResults />;  
}
```

Initially:

Navbar  
Sidebar  
PerformanceResults(example.com)



Then you navigate:

/analyze/example.com



```
/analyze/google.com
```

Next.js doesn't need to conceptually do:

```
destroy Navbar
destroy Sidebar
destroy everything
create Navbar
create Sidebar
create everything
```



Instead, the route tree changes its page segment:

```
Navbar
Sidebar
PerformanceResults(google.com)
```

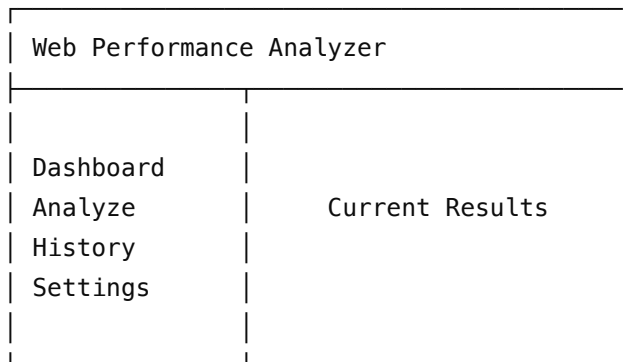


The layout can remain mounted.

## 7. Why is this useful?

Think about something like your analyzer.

Your layout could contain:



You don't want this entire shell to disappear and be recreated every time the analyzed domain changes.

Instead:

```
Shell
├─ Navbar      ← stays
├─ Sidebar     ← stays
└─ Page       ← changes
```



That's one of the big benefits of nested layouts.

---

## 8. But wait — doesn't the server still have to do work?

Yes!

And this connects directly to your question from the previous lesson:

"If the server renders something and React hydrates it, doesn't that still take time?"

Exactly.

Client-side navigation **doesn't mean no server work**.

Instead, Next.js can request the **new route's server-rendered result/data**, rather than requesting an entirely new HTML document and doing a complete browser page load.

Conceptually:

Initial visit



Browser



Request document



Next.js server



HTML + React Server Component result



Browser



Hydration

But subsequent navigation can be more like:

Click <Link>



Next.js Router



Request new route's RSC payload



Server renders the changed route segments



Browser receives result



React updates the relevant part

So you're **not eliminating rendering**.

You're reducing **unnecessary document-level work**.

That's the important distinction.

---

## 9. And this is where `<Link>` gets really cool: prefetching

Suppose you're on:

/analyze/example.com



and Next.js sees:

```
<Link href="/analyze/google.com">  
  Google  
</Link>
```

Depending on the situation, Next.js can **prefetch** the route.

Meaning:

```
User is currently reading page  
  ↓  
Next.js notices <Link>  
  ↓  
Fetches information needed for the destination  
  ↓  
Stores it  
  ↓  
User clicks  
  ↓  
Navigation can feel much faster
```



So the user might experience:

```
click  
  ↓  
almost immediate UI change
```



even though Next.js still had to obtain information for the destination.

---

## 10. Compare the three worlds

This is worth burning into your brain:

### Traditional multi-page website

```
<a>  
↓  
Browser navigation  
↓  
GET new document  
↓  
New HTML  
↓  
Entire document replaced
```



### React + Parcel + React Router

```
<Link>  
↓  
Router intercepts  
↓  
history.pushState()  
↓  
React chooses another component  
↓  
No document reload
```



### Next.js App Router

```
<Link>  
↓  
Next Router  
↓  
Prefetched / fetch route information  
↓  
Server Components / route tree  
↓  
Only necessary route segments update  
↓  
Layouts can remain mounted
```



That's the mental model I want you to have.

---

## 11. One subtle but VERY important thing

Don't think:

**"Next.js navigation = server rendering every time."**

And don't think:

**"Next.js navigation = everything happens in the browser."**

Neither is correct.

It's a **hybrid system**.

The server can handle:

Server Components  
Data fetching  
Rendering server UI



while the browser handles:

Interactions  
Client Components  
State  
Event handlers  
Router behavior  
DOM updates



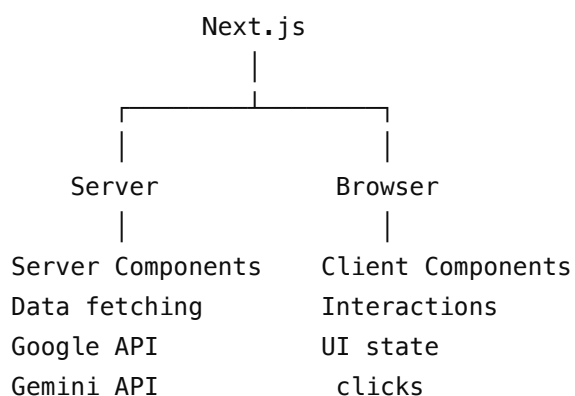
That's why Next.js isn't simply:

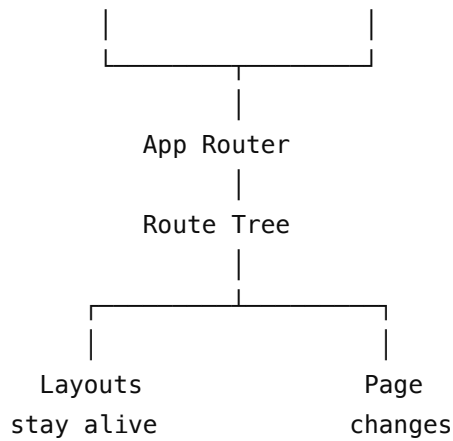
"React but with SSR."

It's a different rendering/navigation architecture built around React Server Components + the App Router.

## Your mental picture

For your project, think:





And **that** is why the distinction matters.

### Next up: Lesson 3.4

Before we jump into data fetching, there's one last piece I want you to understand:

`useRouter` , `usePathname` , `useSearchParams` , **dynamic routes**, and **route params** — especially the difference between:

`/analyze/google.com`



and:

`/analyze?url=google.com`



That will make the URL architecture for your Performance Analyzer much clearer. 🚀



i am still confused between react+parcel router and next js link router for me both looks same

YES 😊 — and honestly, **your confusion is completely justified**.

At the level you're looking at it, they **ARE** very similar.

The mistake is trying to understand Next.js `<Link>` as if it's a completely different concept from React Router. It isn't.

Let's strip away the Next.js terminology for a second.

## The one thing both are doing

You have:

/analyze/example.com



and click:

Analyze Google



Both systems can prevent the browser from doing:

FULL PAGE RELOAD



and instead do:

CHANGE URL

↓

CHANGE WHAT REACT SHOWS



So your instinct is correct:

**"Wait... aren't they basically the same?"**

**Yes — from the user's perspective, absolutely.**

**The difference is what happens underneath the router after the click.**

---

## Let's make it stupidly simple

Imagine you have:

React + Parcel



with React Router.

You might have:

```
<Link to="/analyze/google.com">
  Google
</Link>
```

Click it.

React Router basically says:

"Don't let the browser reload. I'll change the URL and tell React that we're now at /analyze/google.com ."

Then React renders the appropriate component.

```
CLICK
  ↓
React Router
  ↓
URL changes
  ↓
React renders new component
```



Now Next.js:

```
<Link href="/analyze/google.com">
  Google
</Link>
```

Click it.

Next's router says:

"Don't do a full browser navigation. I'll handle this route transition."

Then Next can retrieve the **new route's Server Component output/data** and merge the changed route segments into the existing React tree.

```
CLICK
  ↓
Next Router
  ↓
URL changes
  ↓
Get new route's server component result
  ↓
React updates changed route segment
```

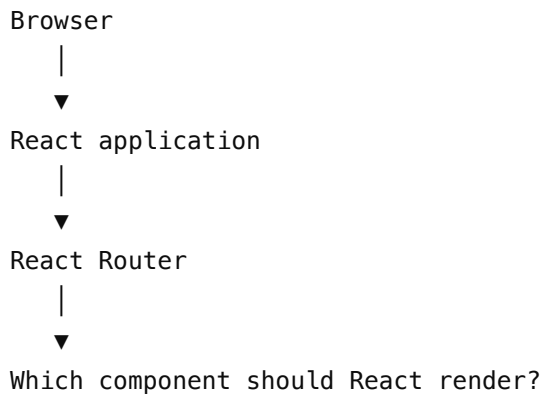


## So where is the REAL difference?

Here's the key.

### React + Parcel

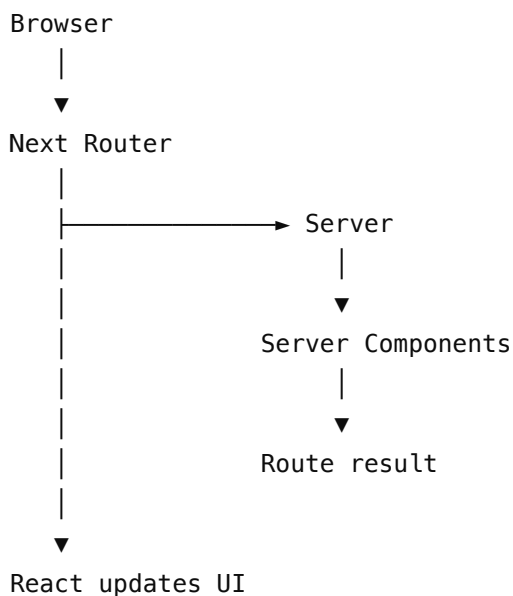
Your application is essentially:



The router is primarily helping your **client-side React application** decide what to display.

### Next.js

Your application is:



The router is coordinating navigation across **client + server**.

## Here's the analogy I think will finally click

Imagine two restaurants 🍔 .

## Restaurant A — React + Parcel

You walk in and say:

"I want the pizza."

The waiter says:

"Okay."

And the kitchen is **already inside the restaurant**.

```
You
↓
Waiter
↓
Kitchen inside restaurant
↓
Pizza
```



---

## Restaurant B — Next.js

You walk in and say:

"I want the pizza."

The waiter says:

"Okay."

But some of the kitchen work happens **in another room/server**.

```
You
↓
Waiter
↓
Kitchen/server
↓
Pizza information
↓
Restaurant
↓
You
```



The waiter is still a waiter in both cases.

**That's the router.**

The difference is what the router is coordinating.

---

## And here's the part that causes 90% of the confusion

You may be thinking:

"But React Router can also fetch data!"

**Correct.**

You may think:

"React Router can also preserve layouts!"

**Correct.**

You may think:

"React Router can also do client-side navigation!"

**Correct.**

You may think:

"So why is Next `<Link>` special?"

**It isn't special because it is a `<Link>` .**

That's the important realization.

`<Link>` **is just the navigation interface.**

The bigger difference is the **framework architecture around it.**

---

## Think of it this way

|                           | React + Parcel + React Router | Next.js |
|---------------------------|-------------------------------|---------|
| <code>&lt;Link&gt;</code> | Yes                           | Yes     |
| Client-side navigation    | Yes                           | Yes     |

|                                           | React + Parcel + React Router   | Next.js          |
|-------------------------------------------|---------------------------------|------------------|
| URL changes without full reload           | Yes                             | Yes              |
| Can preserve UI/layout                    | Yes                             | Yes              |
| Can fetch data                            | Yes                             | Yes              |
| Router                                    | React Router                    | Next Router      |
| Server Components built in                | ✗                               | ✓                |
| Server-side routing architecture built in | ✗                               | ✓                |
| Server/client component boundary          | You build architecture yourself | Next provides it |

So if your question is:

**"Does clicking both links avoid a full page reload?"**

**Yes.**

**"Do both change the URL and update React?"**

**Yes.**

**"Can both preserve parts of the UI?"**

**Yes.**

**"Then why does Next.js feel different?"**

Because **Next** isn't merely giving you a router.

It's giving you an entire application architecture where the router understands things like:

Layouts  
Server Components  
Client Components  
Server-side data  
Route segments  
Streaming  
Prefetching



# One correction to your original mental model

Earlier we had:

Next <Link>



Browser

↓

Next Router

↓

Fetch route data

↓

Update route tree

↓

Preserve layouts

↓

Update page

That's useful, **but don't memorize it as "Next Link does this, React Router doesn't."**

That's the trap.

Instead memorize:

React Router:



Browser

↓

React Router

↓

Client React tree changes

and:

Next Router:



Browser

↓

Next Router

↓

May coordinate with server

↓

React tree changes

That's the meaningful distinction.

---

**And THIS is why I wanted you to understand this before data fetching**

Your Performance Analyzer will eventually have something like:

/analyze/google.com



You might click:

/analyze/github.com



The navigation itself is **not the interesting part**.

The interesting question becomes:

**Where does the data for GitHub's analysis come from?**

For example:

Click GitHub  
↓  
Next Router  
↓  
New route  
↓  
Server Component  
↓  
Google PageSpeed API  
↓  
Gemini  
↓  
Results  
↓  
Browser



And **that** is where Next.js starts giving you something substantially different from the basic React + Parcel setup you were using.

So I wouldn't spend too much more brainpower trying to find a huge difference between the two `<Link>` components.

**Your takeaway should simply be:**

**React Router and Next Router both solve client-side navigation. Next's router is integrated with Next's server/client rendering architecture, so navigation can involve changing only route segments and obtaining Server Component results from the server.**

That's it. You've actually understood the confusing part correctly. 🤔

Now we can move on to **Lesson 4 — Data Fetching**, where the difference becomes much more concrete.



Before we jump into data fetching, there's one last piece I want you to understand:

`useRouter ****`, `usePathname ****`,  
`useSearchParams ****`, **dynamic routes**, and **route params** — especially the difference between:

`/analyze/google.com`

and  
[Show more](#) ▾

Exactly 👍 Let's do that **last piece of Lesson 3** before data fetching. And I'll keep it tied to your Performance Analyzer rather than drowning you in Next.js terminology.

## Lesson 3.4 — URLs, Dynamic Routes & Router Hooks

There are **two different ways** you could represent the same analysis:

### Option A — Dynamic route

`/analyze/google.com`



### Option B — Query/search parameter

`/analyze?url=google.com`



They look similar, but Next.js treats them differently.

---

## 1. First understand the URL itself

Take:

```
https://myapp.com/analyze/google.com?device=mobile
```



Break it apart:

```
https://myapp.com
```

```
└─ domain
```

```
/analyze/google.com
```

```
└─ route parameter  
└─ route
```

```
?device=mobile
```

```
└─ search parameter
```



So:

```
/analyze/google.com
```



and:

```
/analyze?url=google.com
```



are **not the same type of URL**.

---

## 2. Dynamic route: /analyze/google.com

In Next.js App Router, you could create:

```
app/
```

```
└─ analyze/  
    └─ [domain]/  
        └─ page.tsx
```



The [domain] means:

"Whatever appears in this part of the URL, treat it as a dynamic value."

So:

```
/analyze/google.com
```



gives:

```
domain = "google.com"
```



And:

```
/analyze/github.com
```



gives:

```
domain = "github.com"
```



And:

```
/analyze/example.com
```



gives:

```
domain = "example.com"
```



Same page file:

```
[domain]/page.tsx
```



Different parameter.

---

### 3. How does the page receive it?

In a Server Component page, conceptually:

```
export default async function Page({ params }) {  
  const { domain } = await params;  
  
  return <h1>Analyzing {domain}</h1>;  
}
```

So when the URL is:

```
/analyze/google.com
```



you get:

```
domain = "google.com"
```



Your page can then use that domain to call your API.

For your project:

```
/analyze/google.com
  ↓
domain = google.com
  ↓
PageSpeed API
  ↓
Performance data
  ↓
Gemini
  ↓
Analysis
```



That's a very clean architecture.

---

## 4. Now the other option: `/analyze?url=google.com`

Here you don't have a dynamic route.

Your folder could simply be:

```
app/
├── analyze/
│   └── page.tsx
```



The URL is:

```
/analyze?url=google.com
```



The route itself is just:

```
/analyze
```



and:

```
url=google.com
```



is a **search parameter**.

You can think:

/analyze



= which page?

?url=google.com



= extra information given to that page

---

## 5. This is where useSearchParams() comes in

In a Client Component:

```
"use client";

import { useSearchParams } from "next/navigation";

export default function Analyze() {
  const searchParams = useSearchParams();

  const url = searchParams.get("url");

  return <h1>Analyzing {url}</h1>;
}
```

For:

/analyze?url=google.com



you get:

url = "google.com"



---

## 6. So what is usePathname() ?

This one is much simpler.

It tells you:

"What path is currently in the URL?"

For:

```
/analyze/google.com
```



you get:

```
/analyze/google.com
```



For:

```
/analyze?url=google.com
```



you get:

```
/analyze
```



Notice:

```
/analyze?url=google.com
```

↑

usePathname() does NOT give you this part



It gives only:

```
/analyze
```



The `?url=google.com` portion is handled by `useSearchParams()` .

## 7. And useRouter() ?

This one is different.

`useRouter()` is mainly for **telling Next.js to navigate somewhere programmatically.**

For example:

```
"use client";

import { useRouter } from "next/navigation";

export default function Button() {
  const router = useRouter();

  function analyze() {
```

```
    router.push("/analyze/google.com");
  }

  return (
    <button onClick={analyze}>
      Analyze
    </button>
  );
}
```

Click:

Analyze



and your code says:

router.push(...)



which navigates to:

/analyze/google.com



## 8. So remember these three like this 🧠

Hook

Question it answers

useRouter()

"Can I make the app navigate?"

usePathname()

"What path am I currently on?"

useSearchParams()

"What query/search values are in the URL?"

Example:

/analyze/google.com?device=mobile



You could conceptually have:

```
usePathname()
  ↓
/analyze/google.com

useSearchParams()
  ↓
```



```
device = mobile

params
  ↓
domain = google.com

useRouter()
  ↓
navigate somewhere
```

## 9. Now let's make your Performance Analyzer

Suppose the user has an input:

```
Enter website
google.com

[ Analyze ]
```

You have two possible architectures.

### Architecture A

After clicking:

```
/analyze/google.com
```

Folder:

```
app/
├─ analyze/
│   └─ [domain]/
│       └─ page.tsx
```

And:

```
URL
  ↓
dynamic route
  ↓
domain
  ↓
analysis
```

## Architecture B

After clicking:

```
/analyze?url=google.com
```



Folder:

```
app/  
└─ analyze/  
    └─ page.tsx
```



And:

```
URL  
↓  
search parameter  
↓  
url  
↓  
analysis
```



---

## 10. Which one would I choose for your project?

I'd lean toward:

```
/analyze/google.com
```



for the **main analysis page**.

Why?

Because the thing you're displaying is fundamentally:

**"The analysis page for this particular domain."**

So the domain is part of the resource you're viewing.

It also gives you nice URLs:

```
/analyze/google.com  
/analyze/github.com  
/analyze/amazon.com
```



rather than:

```
/analyze?url=google.com  
/analyze?url=github.com  
/analyze?url=amazon.com
```



Both are technically valid, though. This isn't a case where one is universally "correct."

---

## 11. But there's an important catch with domains ⚠️

A domain can contain characters that have special meaning in URLs.

For example:

```
https://www.google.com
```



If you're putting an entire URL into a route parameter, you have to think about URL encoding.

That's why you might actually choose:

```
/analyze?url=https%3A%2F%2Fwww.google.com
```



if your input is an **entire URL** rather than simply:

```
google.com
```



That's an architectural decision we'll make when we build your analyzer.

---

## 12. One final distinction: params vs searchParams

This is probably the most important thing to remember from this lesson.

### Dynamic segment

```
/analyze/google.com
```



comes from:

```
[domain]
```



and is accessed through:

`params`

## Search parameter

`/analyze?url=google.com`

comes from:

`?url=...`

and is accessed through:

`searchParams`

So:

`/analyze/google.com`

|  
└─ params

`/analyze?url=google.com`

|  
└─ searchParams

That's the distinction.

## Your Lesson 3 mental map

At this point, I'd keep this tiny cheat sheet:

`<Link>`

↓

`Navigate declaratively``useRouter()`

↓

`Navigate programmatically``usePathname()`

↓



"What path am I on?"

useSearchParams()

↓

"What ?key=value values are present?"

[domain]

↓

Dynamic route segment

params

↓

Value from [domain]

?url=google.com

↓

Search parameter

searchParams

↓

Value from ?url=

And now you're genuinely ready for **Lesson 4 — Data Fetching in Next.js**.

That's where we'll connect the dots to your actual project:

/analyze/google.com

↓

route param

↓

Server Component

↓

Google PageSpeed Insights API

↓

performance data

↓

Gemini

↓

AI explanation

↓

UI



That is where Next.js starts becoming much more interesting than just "React with a different router." 🚀



ChatGPT can make mistakes. Check important info.